PEAKSKILLS

MVP PRODUCT, ARCHITECTURE & DESIGN SPECIFICATION

Antigravity Build Document — Agreed MVP Scope

Product positioning: AI-powered Talent & Opportunity Intelligence Platform

MVP focus: AI Talent Matching Engine for a mixed talent pool of students and professionals/job seekers.

Core promise: Employer enters hiring requirements → PeakSkills analyzes eligible talent → returns the Top 10 best-fit candidates with an explainable match score.

Important: This document defines ONLY the agreed MVP. Do not build future ecosystem modules unless explicitly requested in a later phase.

1. MVP SCOPE

The MVP must be a functional web application, not a static prototype. It must support authentication, database persistence, candidate data, employer vacancies, matching, ranking and explainable AI results.

Core MVP modules

• **Candidate/Talent database:** Students + Professionals/Job Seekers.

• **Candidate profiles:** education, skills, languages, experience, achievements, CV and preferences.

• **Employer accounts:** company profile and vacancy management.

• **Vacancy creation:** employer defines position, requirements, skills, experience, education, languages and preferences.

• **AI job understanding:** converts natural-language vacancy descriptions into structured requirements.

• **AI/CV intelligence:** extracts structured information from CVs where available.

• **Matching engine:** hard requirements + structured scoring + semantic similarity.

• **Top 10 ranking:** best-fit eligible candidates.

• **Why Recommended:** concise, evidence-based explanation of the match.

• **Candidate-side job matching:** candidate sees relevant vacancies and match scores.

• **Admin/seed environment:** synthetic candidates, employers and vacancies for demonstration/testing.

• **Security foundation:** role-based access, server-side AI calls, validation, audit-friendly architecture.

Explicitly OUT OF MVP

AI Live Interview • Exchange/Master's Agent • Scholarship Agent • Sponsorship Agent • University ERP/API integrations • Predictive hiring probability • Career Growth Engine • Mobile app • Complex microservices • Custom ML training pipeline.

2. USER ROLES & ACCESS

Role	Primary actions	Access
Candidate	Create profile, upload CV, view matched jobs, see match explanations	Own profile + public/job-relevant data
Employer	Create company, create vacancies, view ranked candidates, shortlist	Own company/vacancies + authorized candidate information
Admin	Manage seed data, inspect records, manage taxonomy, monitor matching	Full controlled access

Candidate type

The database must support **STUDENT** and **PROFESSIONAL** under one universal Candidate/Talent model. Do not build two separate incompatible systems.

3. TECHNICAL ARCHITECTURE

Recommended MVP stack

• **Frontend:** Next.js + TypeScript.

• **UI:** Tailwind CSS + shadcn/ui or an equivalent accessible component system.

• **Backend:** Next.js server-side APIs/server actions with clear service boundaries.

• **Database:** PostgreSQL.

• **Vector search:** pgvector.

• **Authentication:** Supabase Auth or an equivalent secure managed auth system.

• **Storage:** Supabase Storage or equivalent private object storage for CV files.

• **AI:** provider abstraction supporting Gemini/OpenAI without hard-coding the application to one provider.

• **Validation:** Zod or equivalent schema validation.

• **Deployment:** Vercel + managed PostgreSQL/Supabase is acceptable for MVP.

High-level flow

Employer → Vacancy → AI Job Parser → Hard Eligibility Filters → Structured Scoring + Semantic Matching → Ranking → Top 10 → Explanation → Shortlist

Candidate flow: **Candidate Profile/CV → Structured Profile → Job Matching → Ranked Opportunities → Match Explanation → Apply/Interest**.

Do NOT allow the LLM alone to decide ranking. Deterministic requirements and scoring must remain auditable.

4. DATABASE / DATA MODEL

Core tables

users — id, email, role, created_at

candidates — id, user_id, candidate_type, name, headline, bio, location, availability, expected_salary, profile_completion

candidate_education — candidate_id, institution, degree, field_of_study, year, GPA, graduation_year

skills — id, name, category

candidate_skills — candidate_id, skill_id, proficiency

candidate_languages — candidate_id, language, level, certification

experiences — candidate_id, company, position, start_date, end_date, description

achievements — candidate_id, title, description

cvs — candidate_id, file_url, parsed_text, parsed_status

employers — id, company_name, industry, size, website, description

vacancies — id, employer_id, title, description, location, employment_type, experience_min, experience_max, salary_min, salary_max, status

vacancy_skills — vacancy_id, skill_id, required, weight

matches — vacancy_id, candidate_id, match_score, skill_score, experience_score, education_score, language_score, semantic_score, explanation, created_at

Data principle

Keep structured fields relational. Use JSON only for flexible metadata. CV files are stored privately; parsed text is stored separately for AI processing. All candidate/employer relationships must be indexed and queryable.

5. MATCHING ENGINE — CORE IP FOUNDATION

The MVP matching engine is a hybrid system.

• **Layer 1 — Hard filters:** mandatory degree, minimum experience, mandatory language/certification, location/work eligibility where applicable.

• **Layer 2 — Structured scoring:** skills, relevant experience, education/major, languages and academic profile.

• **Layer 3 — Semantic similarity:** embeddings compare vacancy meaning with candidate profile/CV meaning.

• **Layer 4 — LLM explanation:** produces a concise evidence-based explanation using only matched profile data.

Initial scoring weights

Skills 35% • Experience 20% • Education/Major 15% • Languages 10% • Academic Profile 10% • Semantic Fit 10%

Weights must be configurable in one service/module, not scattered through the codebase. Later, real hiring outcomes can be used to calibrate the weights.

Important product language

Display **Profile Match** or **Match Score**. Never call it a probability of being hired. A 96% match means profile fit against the configured requirements, not guaranteed hiring success.

6. AI SERVICES

AI must be isolated behind a provider/service abstraction.

Job Parser — Natural-language vacancy → structured requirements

CV Parser — CV → education, skills, experience, languages, certifications

Semantic Embedding — Candidate/vacancy representations for vector similarity

Match Explanation — Evidence-based 'Why recommended?' explanation

AI safety rules

AI output must be validated against schemas. Never expose raw model output directly to the UI. Never let an LLM invent candidate qualifications. Explanations must reference stored candidate/vacancy data.

Provider abstraction

Create an interface such as **AIProvider** so Gemini/OpenAI can be switched without changing business logic. API keys remain server-side and must never be exposed to browser code.

7. PRODUCT UX / DESIGN SYSTEM

Design direction

Premium B2B AI SaaS. Clean, intelligent, credible and restrained. The product should feel closer to a modern enterprise intelligence platform than a generic job board.

Visual language

• Light-first interface with strong whitespace and high readability.

• Use a dark navy/charcoal primary text system with a restrained blue/indigo accent.

• Cards should be clean, lightly bordered and minimally shadowed.

• Match scores can use visual progress/ring indicators, but avoid excessive gradients or gaming-like UI.

• Use clear status badges: Eligible, Strong Match, Shortlisted, etc.

• Typography: modern sans-serif, strong hierarchy, compact enterprise dashboards.

• Responsive desktop-first dashboard, fully usable on tablet/mobile widths.

• Accessibility: sufficient contrast, keyboard navigation, clear focus states and semantic controls.

Core screens

Landing → **Login/Register** → **Candidate Dashboard** → **Candidate Profile** → **Job Matches** → **Employer Dashboard** → **Create Vacancy** → **Match Results** → **Candidate Detail** → **Admin**

8. KEY SCREEN BEHAVIOR

Employer Dashboard

Show: Active Vacancies, Candidate Pool, Match activity, Shortlisted candidates. Primary CTA: **Create Vacancy**.

Create Vacancy

Fields: Position, description, skills, required/preferred skills, experience range, education, languages, location, employment type, salary range and optional preferences. Include **AI Enhance Requirements** to parse free text into structured fields.

Match Results

Show ranked candidates with: rank, match score, candidate type, headline, key strengths, key gaps and **Why Recommended**. Filters: student/professional, location, experience, skills. CTA: View Profile / Shortlist.

Candidate Dashboard

Show profile completion, recommended jobs, match scores, skill gaps and recent opportunities. Keep the experience simple: the candidate should immediately understand **what fits me and why**.

9. DEMO / SEED DATA

The MVP must not open with an empty database. Create a clearly labeled **Demo Environment** with synthetic data.

Recommended initial seed

• **200 candidates:** 100 students + 100 professionals.

• **50 employers:** synthetic companies across IT, telecom, finance, banking, retail, engineering, consulting and other relevant sectors.

• **100 vacancies:** realistic roles across the same industries.

• **Skills taxonomy:** at least 100 normalized skills with categories.

• **Languages:** Uzbek, English, Russian plus other common languages.

• **Education:** realistic Uzbekistan university/institution names can be used for synthetic demo records, clearly labeled as demo data.

• **Matching coverage:** seed data must intentionally contain strong, medium and weak matches so the ranking can be visually tested.

Seed generator requirements

Provide scripts/actions to reset and regenerate demo data. Data must be deterministic when a seed value is supplied. Do not use real people's personal data.

10. SECURITY, PRIVACY & TRUST

Candidate data is sensitive. Security is part of the MVP, not a future feature.

• Role-based access control: candidate, employer and admin.

• Employer can only access its own vacancies and authorized candidate information.

• Candidate can edit/view only their own private profile data.

• CV files stored in private storage; access through authenticated/temporary URLs.

• AI API keys are server-side only.

• Validate all client input on the server.

• Rate-limit AI and expensive endpoints.

• Audit important actions: vacancy creation, candidate view, shortlist and sensitive data access.

• Do not expose unnecessary personal information in search results.

• Separate demo/synthetic data from any future production data.

• AI recommendation is decision support; final hiring decision remains with the employer.

Bias control

Do not use protected or irrelevant personal attributes for ranking. Matching should focus on job-relevant education, skills, experience, languages and other explicitly job-related criteria.

11. API / SERVICE BOUNDARIES

Suggested service modules

• **authService** — authentication and role checks.

• **candidateService** — candidate profiles, education, skills, experience and CV metadata.

• **employerService** — company profiles.

• **vacancyService** — vacancy CRUD and requirements.

• **aiService** — parsing, embeddings and explanations.

• **matchingService** — eligibility, scoring, ranking and match persistence.

• **searchService** — structured + vector search.

• **adminService** — seed data and controlled administration.

Keep business logic out of UI components. Components call services; services access repositories/database; AI calls remain server-side.

12. MVP ACCEPTANCE CRITERIA

The MVP is complete only when the following end-to-end flow works reliably:

1. Admin seeds demo data.

2. Employer registers/logs in.

3. Employer creates a vacancy using structured fields or AI-enhanced free text.

4. System validates mandatory requirements.

5. Matching engine filters ineligible candidates.

6. Eligible students + professionals are scored.

7. Top 10 candidates are ranked.

8. Each candidate has a transparent score breakdown.

9. Each candidate has an evidence-based AI explanation.

10. Employer can open candidate profile and shortlist.

11. Candidate logs in and sees matching vacancies.

12. Candidate can understand why a job matches and identify basic gaps.

13. Data persists after refresh/re-login.

14. Unauthorized users cannot access protected records.

15. AI failures have graceful fallback/error states.

Quality bar

No fake buttons. No dead navigation. No placeholder screens in the core flow. Every core action must produce a real database change or real computed result.

13. ANTIGRAVITY BUILD ORDER

1. Initialize Next.js/TypeScript project, environment variables, linting and folder conventions.

2. Implement database schema, migrations, indexes and seed framework.

3. Implement authentication and role-based route protection.

4. Build Candidate/Talent CRUD and profile pages.

5. Build Employer CRUD and dashboard.

6. Build Vacancy CRUD and requirements model.

7. Build deterministic matching service with hard filters and scoring.

8. Add pgvector embeddings and semantic similarity.

9. Add AI job parser and CV parser behind AIProvider abstraction.

10. Add AI explanation service.

11. Build ranked Top 10 results UI and candidate detail.

12. Build candidate-side job matching.

13. Add admin seed/reset tools.

14. Implement security, validation, rate limiting, logging and error states.

15. Run end-to-end tests and fix edge cases.

16. Polish visual system and responsive behavior.

Do not skip directly to visual polish before the end-to-end matching flow works.

14. FUTURE-READY, BUT NOT BUILT NOW

The database and service architecture should leave extension points for:

• AI Live Interview

• Education Intelligence Agent — Exchange / Master's / Scholarships

• Career Opportunity Agent — Internship / Sponsorship / Early Hiring / Jobs

• University integrations and verified academic data

• Career Growth Engine

• Outcome-based predictive talent models

These must NOT appear as unfinished features in the MVP UI. Keep them out of navigation unless implemented.

FINAL PRODUCT RULE

Build a small product that works end-to-end, not a large product that only looks complete.

The MVP's single most important proof is: **Employer requirements → PeakSkills → Top 10 relevant students + professionals → explainable match.**

Once that loop works with real pilot employers, expand the intelligence layer.